

JAVA

java.util.*;

__/__/

this() -> constructor

Test get() & return this; } , this.a = a; Test () of this(a,b); } test(int a, int b) & }

initializes block -> of Source ("Common Part of all Constructors"); }

STRING [Immutable]

String str0 = "xyz"; ← [String literal, Pool]
String str1 = "xyz"; (str == str1) true

length(), charAt(), String.format("%"), substring(begin, end),
contains(), equals, equalsIgnoreCase(), split(""),
indexOf(ch, begin), isEmpty(), concat, replace(old, new), trim(),
String.valueOf(),
toUpperCase(), toUpperCase(),

for (float ... args) { sum = args[0]; for (int i = 1; i < args.length; i++)
sum += args[i]; }
while (true) { } do { } while (true);

SB -> append(), reverse(), toString(), charAt(), lastIndexOf(),
capacity(), isEmpty(), substring(), length(), indexOf(), delete(),
start, end

INHERITENCE -> Using Class -> Multi level, Single, Hierarchical, Hybrid
(extends)
implements Using Interface -> Multiple

POLYMORPHISM -> Compile time (Static) -> Method Overloading
(Is-a relⁿ) Runtime (Dynamic) -> Method Overriding

(a instance of b), final (constants, Not Inherited, can't override method)
super(), super.method();

$\hookrightarrow$ java.time.LocalDate / local time / local date + time
 $\hookrightarrow$ java.util.Date / calendar ~~date~~ / ~~time~~ / ~~date + time~~

[can't instantiate] Abstract Class [non-access] { abstract class Main d }

- $\hookrightarrow$ can have both abstract & non-abstract method
- but if a class has abs-meth., the class should be abs.
- $\hookrightarrow$ can have param. constructor
- $\hookrightarrow$ must override abs. method in child class.

Abstraction achieved via Interfaces & Abs. Classes

can't instantiate
 X Constructor
 INTERFACES [Complete abs] Implemented by class,
 [Collection of abs. meth.], Multiple Inheritance
 interface ~~and~~ others
 list of interface ~~extends~~ extends

DATE TIME

$\hookrightarrow$ calendar.add(Calendar.DATE, 5), Date fd = cal.getTime()

$\hookrightarrow$ Local Date/Time/DateTime ^{new} [ZonedDateTime] = ~~new~~ (d/t/DT)/Zdt. ^{only} ~~new~~

$\hookrightarrow$ DateTimeFormatter f = DateTimeFormatter.ofPattern("MM-dd-yyyy")

ex- LocalDate pd = LocalDate.parse("2015-01-13", formatter);
 $\hookrightarrow$ pd.plusDays(5); minInstant() isBefore() if after()

NESTED CLASS { static inner { inner class method local inner Anonymous

$\hookrightarrow$ Static nested class $\rightarrow$ associate with outer class, access static members
 $\hookrightarrow$ NS c = new NS(); c.display();

$\hookrightarrow$ Inner class $\rightarrow$ associate with obj of outer, access all members (static, non static)

ex:- Outer.InnerClass obj = outer.new InnerClass();

java.sql.Connection;
DriverManager; /*

__/__/

↳ Local/Method local/^{nested} class : can't have static var, access final/effctive ^{final} member

↳ Anonymous Class :- no name, can use extends for abstract & implement Interfaces

interface Animal { void sound(); }

class Outer { ~~void~~ ^{void} meth() { Animal a = new Animal() {
public void sound() {
sound("Nga");
};
a.sound();
};

to 7/10 # @FunctionalInterface ^{make sure} [Single abso. method] can use lambda exp

40 → 100

Built-In → Runnable ^{Runnable} : r() → sound(); [Lambda ✓]

Comparator<T> : (a, b) → a.compareTo(b);

Predicate<T> : x → x > 0;

Callable

ex:- IISAM @fI
interface name of void of Gfx; } ^{name var = () → sound();}

Types :-

1) Consumer ^(sort), Predicate ^{TF}, Function ^{afine}, Supplier ^{output}

EXCEPTION HANDLING
try { ... } catch (Illegal / SQLException) {
finally { ... }
throw new CustomClass("message");

Checked : Must be handled (IOException)

Unchecked : Don't need explicit handling (Divide by 0, NullPointerException)

Show-Source.getMessage(), e.printStackTrace();

↳ class CustomClass extends Exception/RuntimeException & ~~public void close() { ... }~~
public void meth() throws Exception { Super(message); }

↳ public class Main { psvm() ~~throws~~ { try { ^{throw new} CustomClass(); } }

↳ Assertion :- assert (condition : errorMsg) / age > 0 : "U small";

Nested try & catch

↳ java -ea/dmgsApp

java.io.Serializable;

class name implements Serializable & private String name, int age;
name() & ...

@Override

public String toString() & return "Name" + name + "age: " + age; }

try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("pers.dat/ser"))) {
oos.writeObject(objname); oos.write(~~out~~) // Multiple

try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("pers.dat/ser"))) {
Person per = (Person) ois.readObject(); & // use loop for multiple
catch (ClassNotFoundException e) { } } // description

public class Pair<T, U> & private T first, private U second;
public class Comparable<T>

<? extend T> (if Number & all float, double) UPPER BOUND
<Super T> (if Integer & Integer & Number only) LOWER BOUND

Red-black
Tree

ArrayList<Integer> sc = new ArrayList<>(); sc.add(20); sc.get(1);
TreeSet<Class> Ts = new TreeSet<>(); Ts.add(new Class(30));
↳ override compareTo() &
public int compareTo(Class other) & return this.val.compareTo(other.val);

new Ts<> (Comparator.reverseOrder()); // for datatypes

HashMap<K, V> hs = new HashMap<>(); hs.put(10); // remove(10) / get(10)
for (var entry : hs.entrySet()) & entry.getKey(), entry.getValue(); }
↳ hashCode() & equals()

Deque<E> addFirst(), addLast(), offer(), poll(), peek()

ResultSet rs =

executeUpdate() → DML
Query() → Retrieval

__/__/

try (Connection conn = DriverManager.getConnection(url, user, pass))

DML

PreparedStatement ps = conn.prepareStatement("INSERT INTO ...");
ps.setInt(1, 101), ps.setString(2, "A123"), ps.setDouble(3, 3.14);

int rs = ps.executeUpdate();
Sort ("rs + " rows inserted");

Retrieval

Statement st = conn.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM Score");
while (rs.next()) Sort ("rs.getInt() + " + rs.getString()/Double);